W10

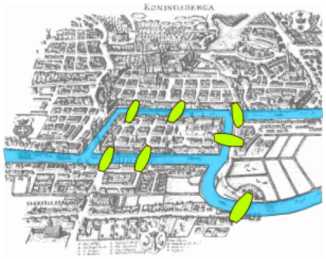
GRAPH

Hsin-Mu Michael Tsai

2026/04/28

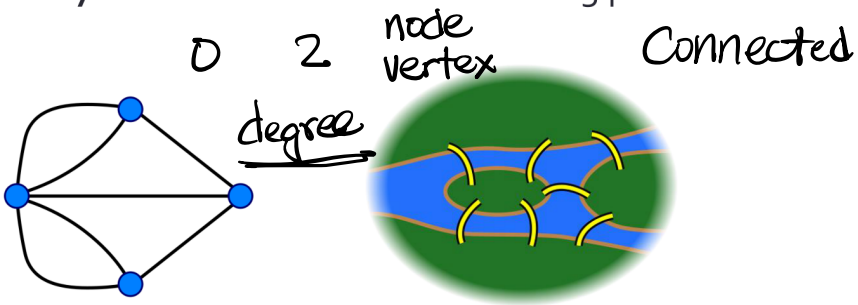
Overview
(BFS)
DFS

Königsberg Seven Bridge Problem



- In 1736, Euler attempted to solve this problem:

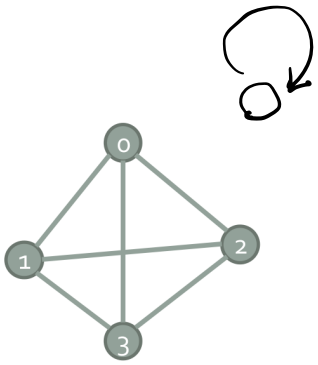
- In the map on the right,, is it possible to find a path, such that every bridge is crossed **exactly once** and returns to the starting point??



Answer: Graph must be connected & must have 0 or 2 node with odd degree

A Graph

- **G**: a graph, consists of two sets, **V** and **E**.
- **V**: a finite, nonempty set of vertices.
- **E**: a set of pairs of vertices.

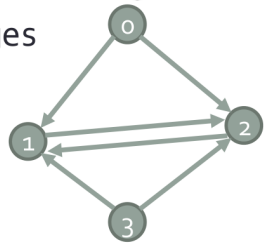


- G=(V,E)
- V={0,1,2,3}
- E={(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)}

Directed & undirected graph

- **Directed graph:** the edges have directions
  Undirected graph: the edges have no directions
- Directed graph is also called **digraph**
- Undirected graph is simply called **graph**

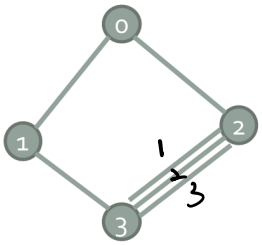
- (1,2) and (2,1) in undirected graph represent the same edge
- <1,2>and<2,1>in digraph represent different edges
- V={0,1,2,3}
- E=?
- E={<0,1>,<0,2>,<1,2>,<2,1>,<3,1>,<3,2>}



Note: In the textbook, digraph also uses () parentheses, but I prefer to use <> angle brackets to represent digraph's edges, which seem clearer.

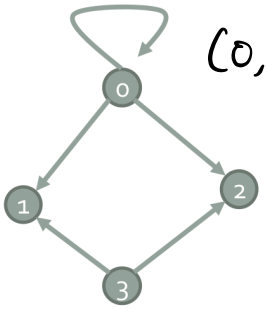
Self Edge & Multigraph

Multigraph



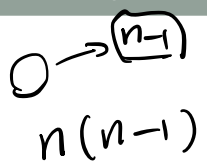
(2,3)$_1$
(2,3)$_2$
(2,3)$_3$

Graph with self-edges <v,v>



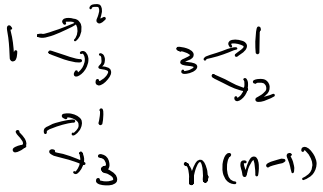
(0,0)

Maximum number of edges

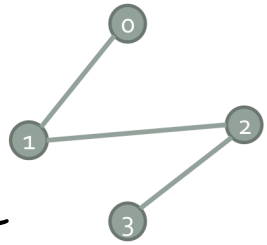


- Suppose a graph has no self edge, nor a multi graph:
  - What is the maximum number of edges in a (undirected) graph with **n vertices**?
  - What is the maximum number of edges in a digraph with **n vertices**?

- Answer: undirected graph: $\frac{n(n-1)}{2}$, digraph: $n(n-1)$



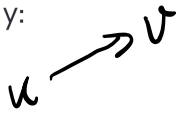
Definitions



- Adjacent (相鄰):
  If there is edge (u,v), then u and v are **adjacent**.
  - u is adjacent to v, v is **adjacent to** u.
  - For directed edge <u,v>, we say v is adjacent to u.
    (but note in some article the opposite definition is used)

u → v

- Incident (作用在):
  - Undirected edge (u,v) —>
    the edge (u,v) is **incident** to u and v. Or we can say:
    - the vertex u is **incident** to v; or
    - the vertex v is **incident** to u.
  - For directed edge <u,v>, we can say the edge **leaves** u and **enters** v.



- Subgraph:
  If G=(V,E), G'=(V',E') is G's subgraph. We have $V' \subseteq V$ and $E' \subseteq E$

V E

G=(V,E)

E={(a1,b1) }

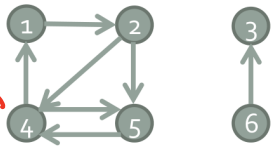
a1∈V

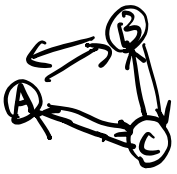
b1∈V

Degree of vertex

- **Degree** of a vertex: the number of edges connected to that vertex
- For a digraph, we have **in-degree** and **out-degree**
  - **in-degree**: the number of edges entering the vertex
  - **out-degree**: the number of edges leaving the vertex
  - **degree**=in-degree + out-degree
- **Isolated** vertex: a vertex with degree=0
- Edge count and degree relationship:

directed graph & undirected graph





$$e = \frac{\left(\sum_{i=0}^{n-1} d_i\right)}{2}$$

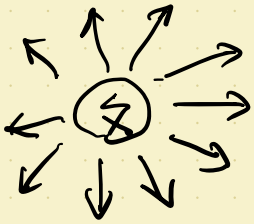
degree

d_i : vertex i degree

$e = |E|$

Digraph:

$$e = \frac{\sum_{0}^{n-1} d_i}{2}$$



out-degree = 10

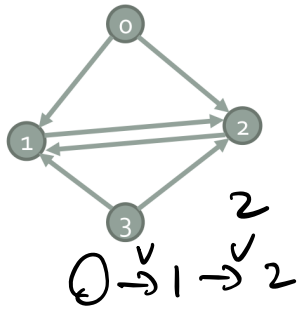
degree = in-deg. + out-deg.

$$e = \frac{4}{2} = 2$$

Path

- **Path**:
  a path from u to v's path, is **a sequence of vertices**, $u, i_1, i_2, \ldots, i_k, v$, and $(u, i_1), (i_1, i_2), \ldots, (i_{k-1}, i_k), (i_k, v)$ are edges.

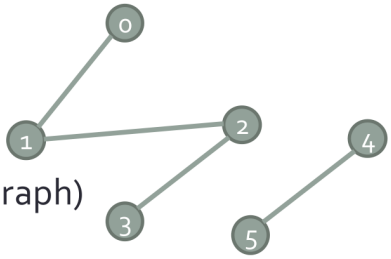
  Similarly for the definition of path in digraph



- If there is a path p from u to u′,
  then we say u′ is **reachable** from u via p.
- The path p can be represented as $u, i_1, i_2, \ldots, i_k, v$.
- The **length** of a path: **the number of edges** in a **path**
- **Simple path**: other than the start and end vertices, no other vertices appear more than once in the path
- **Cycle**: a simple path with the same start and the end vertice
- **Subpath**: a continuous sub-sequence of the vertex sequence of a path

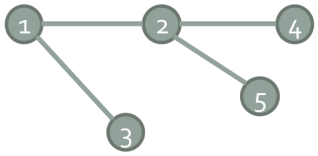
Connected

- **In undirected graph:**
  Vertices u and v are said to be **connected**
  iff there is a path from u to v. (graph & digraph)



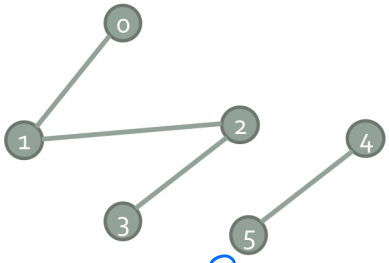
- **Connected graph:**
  G is a connected graph
  iff **every pair** of distinct vertices u and v in V(G) is connected

- **Connected component** (also called directly component):
  A maximal connected subgraph

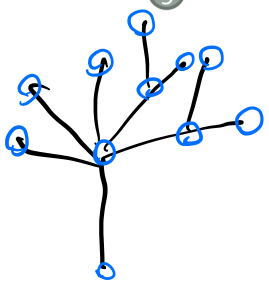


Connected

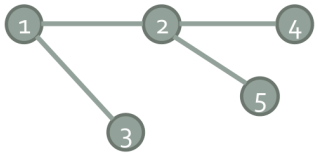
- **Maximal**: no other subgraph in G is both connected and contains the component.



- Q: Tree is a kind of graph?
- A: connected and acyclic (no cycle) graph

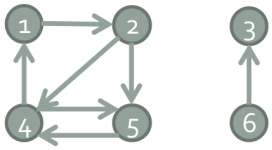


- Q: Forest is a kind of graph?
- A: acyclic graph



Strongly Connected

- **Applicable to digraph**



- **Strongly connected**:
  G is strongly connected
  iff for every pair of distinct vertices u and v in V,
  there is a directed **path from u to v** and also a **directed path from v to u**.

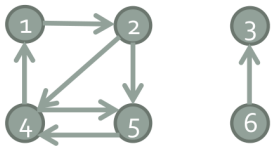
- **Strongly connected component**:
- A **maximal** subgraph which is strongly connected.
- The textbook has an algorithm to decompose a directed graph into strongly connected components in linear time $\Theta(V + E)$

How do we represent a graph?

- Common representations:
  - Adjacency matrix (using array)
  - Adjacency lists (using linked list)

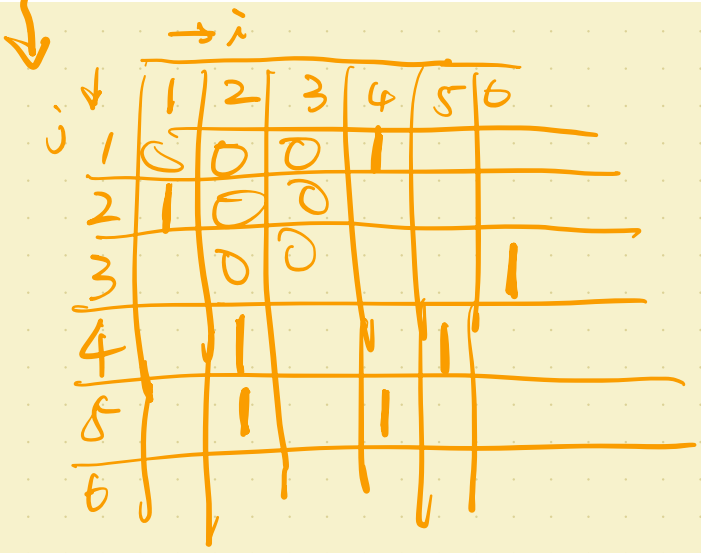
- Both can represent directed & undirected graph

Representation I: Array



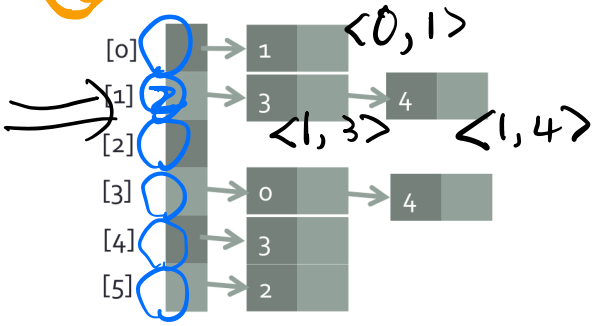
- Adjacency matrix
  - Index treated as vertex number
  - Use matrix value to represent edge
  - Undirected graph: If (i,j) exists, then a[i][j]=1, a[j][i]=1.
  - Digraph: If <i,j> exists, then a[i][j]=1.
- For undirected graph, $a = a^T$ (symmetric along the diagonal)

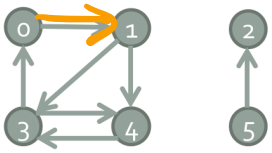
- The total number of edges?
- $O(|V|^2)$
- The consumed space proportional to the number of edges |E|?
- A: No. Usually $|E| \ll |V|^2$. Therefore, adjacency matrix is not very efficient in terms of space.



Representation II: Linked List

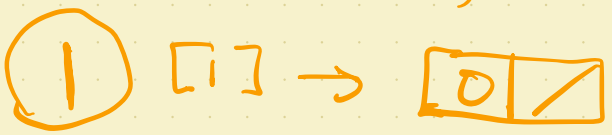
- **Adjacency lists**
- Create a list array a[n], n=|V|
- Each list stores
  the edges leading to other vertex
- Q: How to compute in-degree for all vertices?





- A: Build "inverse adjacency lists" and calculate.

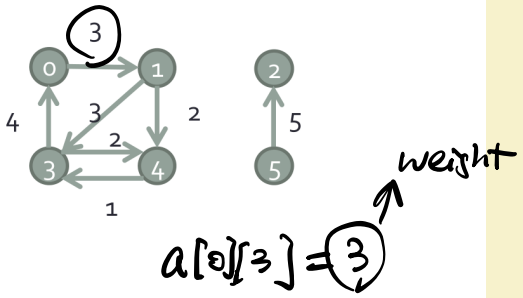
inverse adj list



* what if we don't have inverse adj. lists and still want to calculate in degree for all vertices?

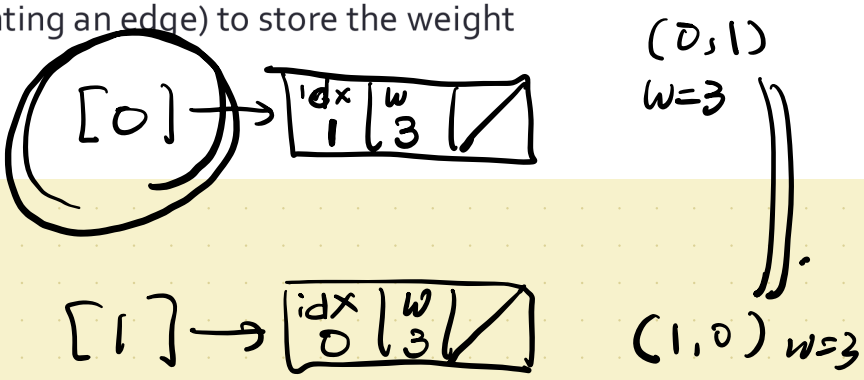
Weighted Edge

- Edge can have **weight,** representing length, or cost
- A graph with weighted edges is also called a **network**

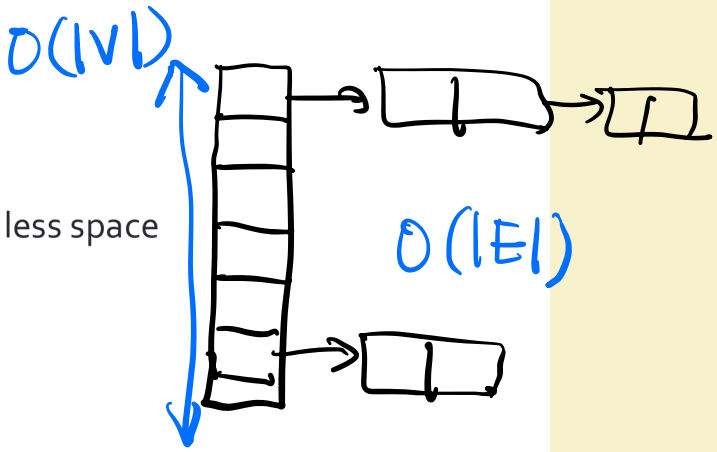


- Q: How to store weights with the introduced representations?

- Adjacency matrix can use array values to store weights
- Adjacency list can use an additional field in each list node (representing an edge) to store the weight



Comparison



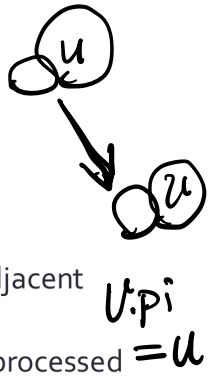
- Adjacency lists:
  - Represent sparse graphs with less space
  - Space: $O(|V| + |E|)$
- Adjacency matrix:
  - Faster to find a specific edge (u,v)
  - Use only 1 bit per edge (for unweighted graph)
  - Aimple and easy, good choice for a small graph
  - Space: $O(|V|^2)$

Depth-first Search DFS

Breadth First Search BFS

- Whenever possible, go deeper → Depth first
- Differences to Breadth-first Search:
  - Produce a forest (multiple trees)
  - Records timestamps:
    discover time (when a vertex turns gray) and
    finish time (when a vertex turns black)
  - Finish time of a vertex is the time when all of its adjacent vertices have been processed.

Data Structure



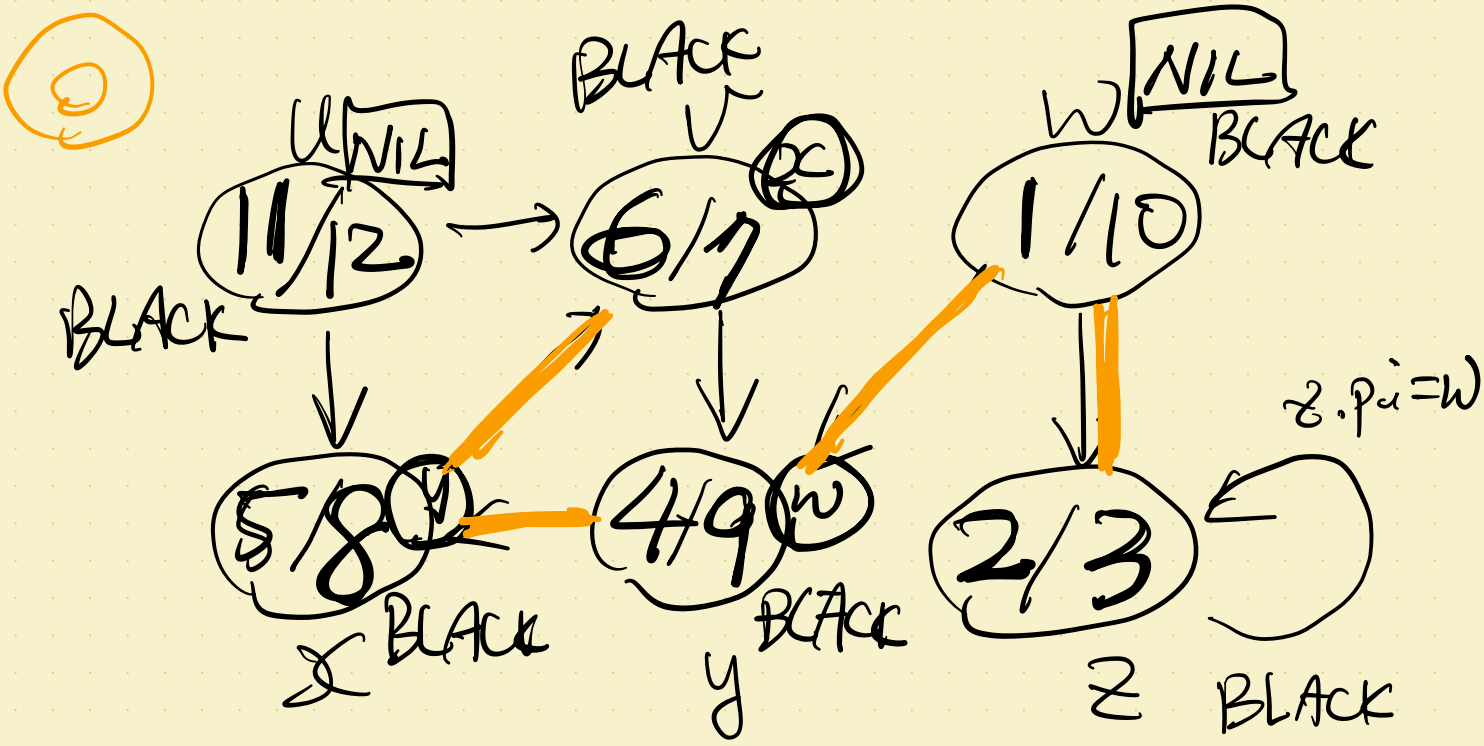
- Each vertex u has the following data structure:
- u.color: record the current vertex's state
  - WHITE: this vertex not yet discover
  - GRAY: this vertex is already discovered, but not all of its adjacent vertices are discovered yet
  - BLACK: this vertex and all its adjacent vertices have been processed
- u.pi: its predecessor vertex
- u.d: discover time (from WHITE→GRAY)
- u.f: finish time (from GRAY→BLACK)

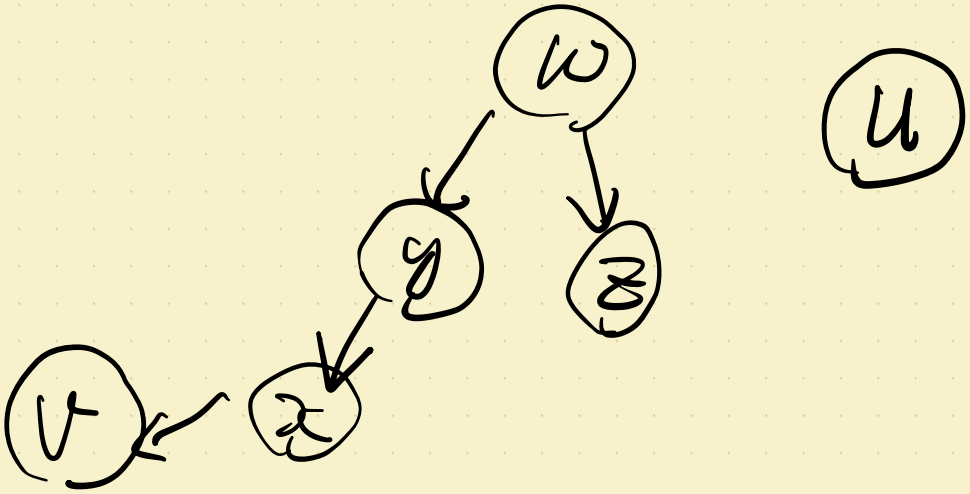
- Timestamps run from 1 to 2|V|
  - Since each vertex will consume two timestamps (discover and finish)

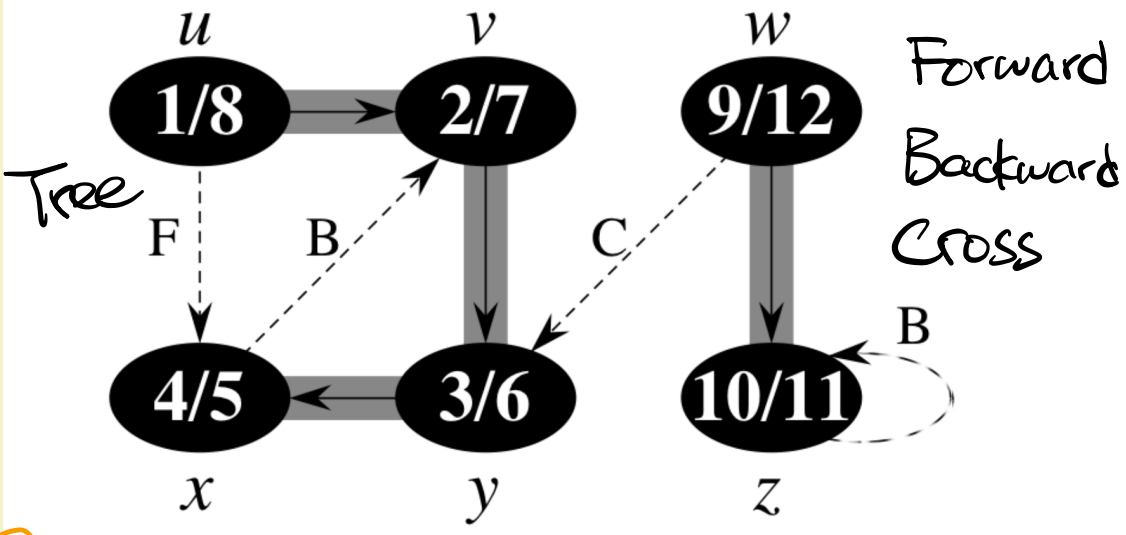
Pseudo-Code

```
DFS(G)
for each vertex u ∈ G.V
  u.color=WHITE
  u.pi=NIL
time=0
for each vertex u ∈ G.V
     if u.color==WHITE
          DFS-VISIT(G,u)
```

```
DFS-VISIT(G,u)
time=time+1
u.d=time
u.color=GRAY
for each v ∈G.Adj[u]
     if v.color==WHITE
          v.pi=u
          DFS-VISIT(G,v)
u.color=BLACK
time=time+1
u.f=time
```







Different ordering of edges in the data structure could result in different DFS results!

If <u,x> is visited before <u,v>, what would happen?

Running Time

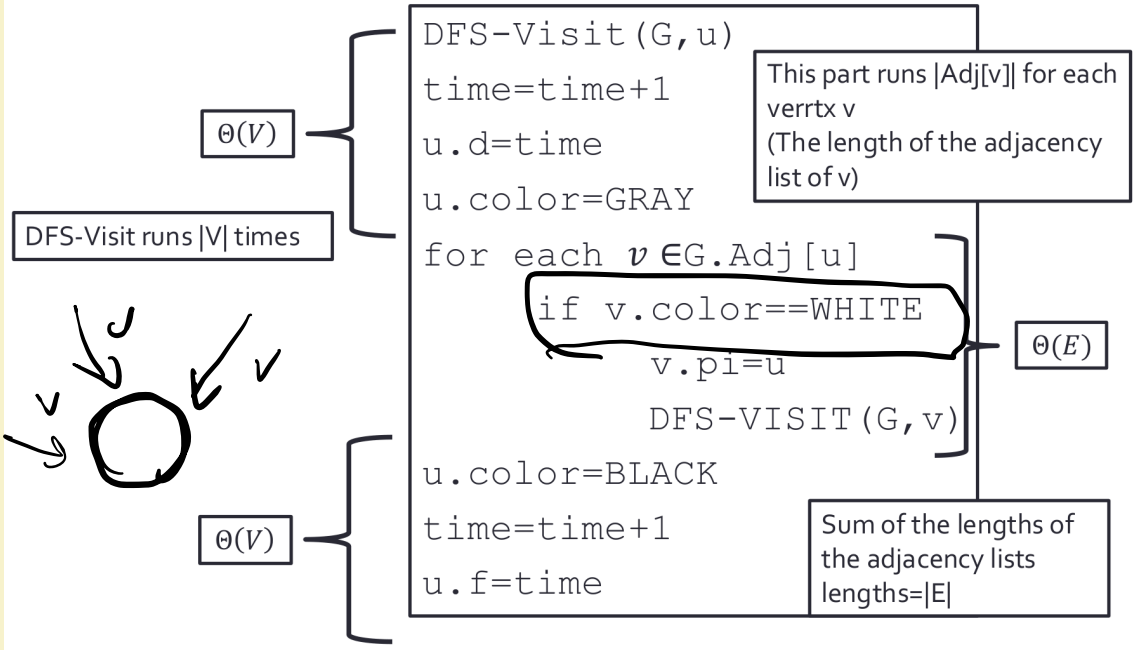
```
DFS(G)
for each vertex u ∈ G.V
  u.color=WHITE
  u.pi=NIL
time=0
for each vertex u ∈ G.V
      if u.color==WHITE
            DFS-VISIT(G,u)
```

$\Theta(V)$

$\Theta(V)$

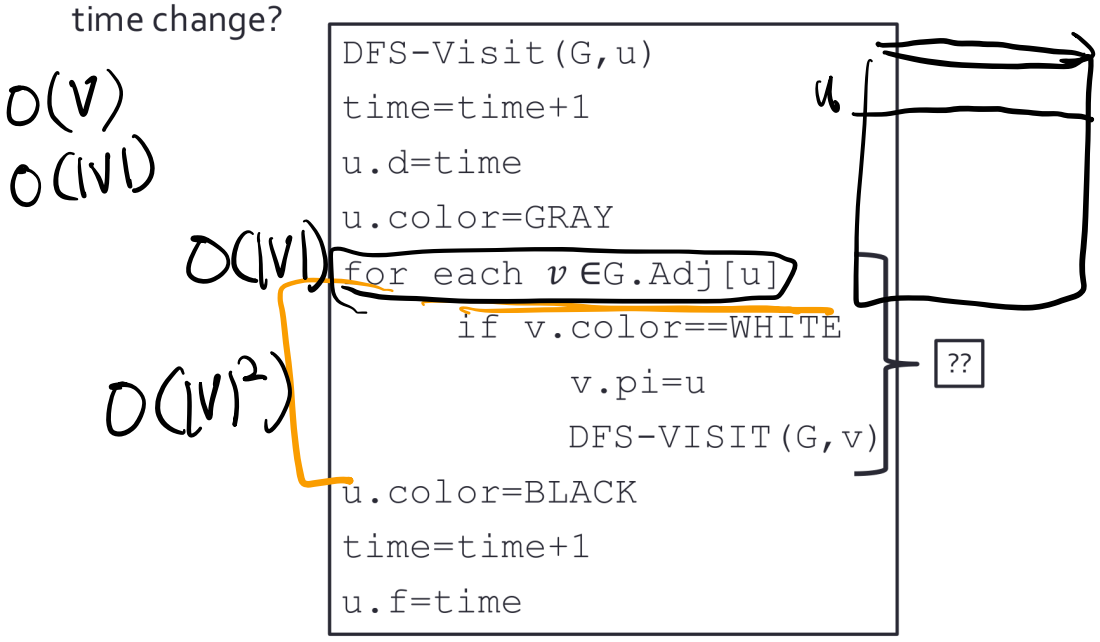
Running

total: $\Theta(V+E)$



Use Adjacency Matrix

- If we switch to use adjacency matrix, how would the running time change?



Depth-first Forest

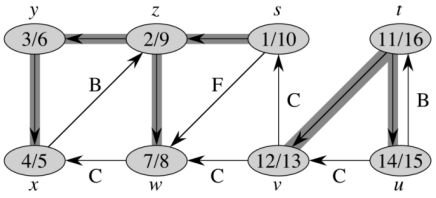
- Similar to breadth-first search, depth-first search can also produce predecessor subgraph:
- $G_\pi = (V, E_\pi)$
- $E_\pi = \{(v.\pi, v) : v \in V \ and \ v.\pi \neq NIL\}$.

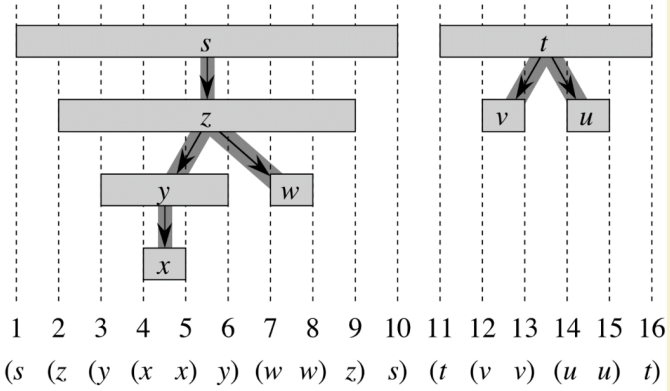
- Predecessor subgraph can produce exactly a forest of depth-first trees, called depth-first forest.

- The edges in E_π is called tree edges.

Parenthesis Structure of DFS

- If we use "(" to represent vertex u's discovery time and ")" to represent vertex u's finish time
- Then the entire DFS search history result in the (and) of a particular vertex to be either contained entirely in a set of (and) of a vertex or completely outside of the (and) of a vertex

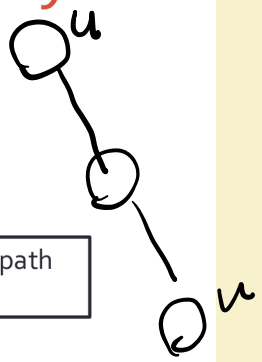




Reading assignment:
See proof in Cormen textbook 20.3

(a) a ( ) b b

White-path (White-path) Property



From the parenthesis structure property:
in depth-first forest of G in, v is u descendant iff u.d<v.d<v.f<u.f.

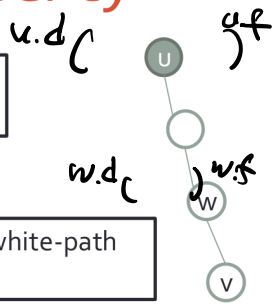
In G's depth-first forest:

v is u's descendant ⟺ When setting u.d, there is a white-path from u to v

- (➔)
  If v = u, then when u.d is assigned, u is still WHITE.
- If v is truly a descendant of u, then u.d < v.d, meaning that v is discovered later.
- At this moment, v must still be WHITE, because we are only assigning u.d.
- Since v can be any descendant of u, this means that every vertex on the path from u to v is also a descendant of u, and therefore should also be WHITE.

White-path (White-path) Property

From the parenthesis structure property:
in depth-first forest of G in, v is u descendant iff u.d<v.d<v.f<u.f.



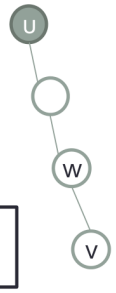
In G's depth-first forest:

v is u's descendant ⟺ When setting u.d, there is a white-path from u to v

- (⇐) Suppose that, at time u.d, there is an all-WHITE path from u to v, but v is not a descendant of u.
- Assume that, on this WHITE path, every vertex before v becomes a descendant of u. In other words, choose v to be the first vertex on the path from u to v that is not a descendant of u.
- Let w be the predecessor of v on this path. Note that u and w may be the same vertex.
- (1) By the Parenthesis Theorem, w.f <= u.f.
- (2) Since there is a WHITE path, u.d < v.d.
- (3) Vertex v will be discovered before w finishes, because there is an edge from w to v. Therefore, v.d < w.f.

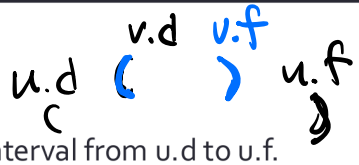
White-path (White-path) Property

From the parenthesis structure property:
in depth-first forest of G in, v is u descendant iff u.d<v.d<v.f<u.f.



In G's depth-first forest:

v is u's descendant ⟺ When setting u.d, there is a white-path from u to v



- Combining the above, we get:
- u.d < v.d < w.f <= u.f
- This means that v.d lies inside the interval from u.d to u.f.
- Therefore, the interval [v.d, v.f] should be contained inside [u.d, u.f].
- This implies that v is a descendant of u, which is a contradiction.

Edges in Depth-first Forest

- There are four types of edges:

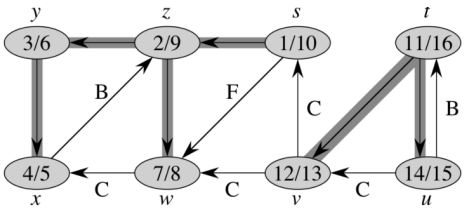
- **Tree edge:**
  An edge in the depth-first forest.
  If vertex v is **discovered through edge (u, v),** then (u, v) is a tree edge.

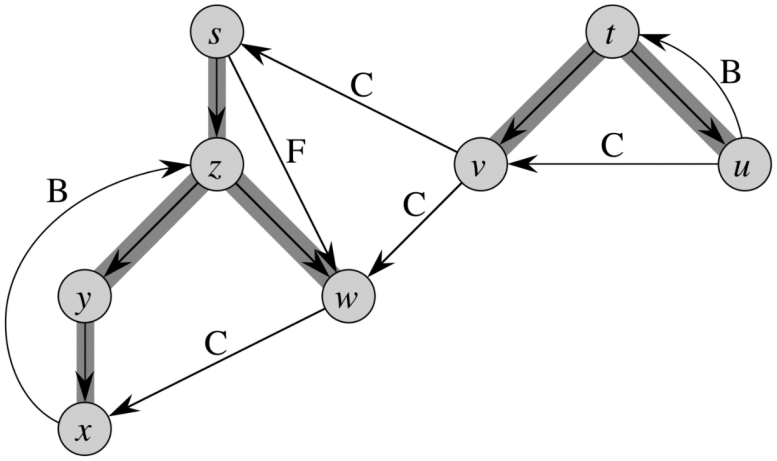
- **Back edge:**
  An edge (u, v) that connects a vertex u to one of its ancestors v.
  A self-loop is also considered a type of back edge.

- **Forward edge:**
  A non-tree edge (u, v) that connects a vertex u to one of its descendants v.

- **Cross edge:**
  All other edges.
  A cross edge may connect two vertices in the same depth-first tree, or it may connect vertices in different depth-first trees.

Example


y
z
s
t
3/6
2/9
1/10
11/16
B
F
C
B
4/5
7/8
12/13
14/15
C
C
C
x
w
v
u


s
t
C
B
z
F
v
C
u
B
C
y
w
C
x

How to identify the type of edge?

- When we first encounter an edge (u, v), the color of v tells us what type of edge it is:

- WHITE → tree edge
  GRAY → back edge
  BLACK → forward edge or cross edge

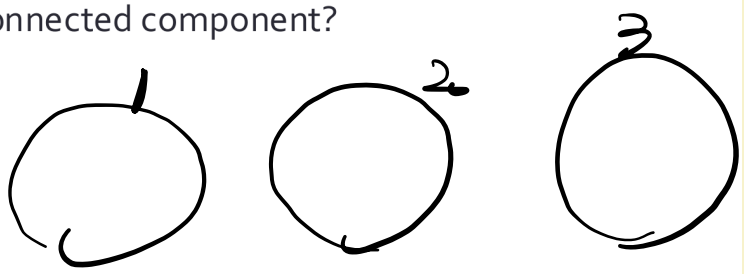
- If u.d < v.d, then it is a forward edge.
  If u.d > v.d, then it is a cross edge.

- In the depth-first forest of an undirected graph, there are no forward edges or cross edges. Think about why?

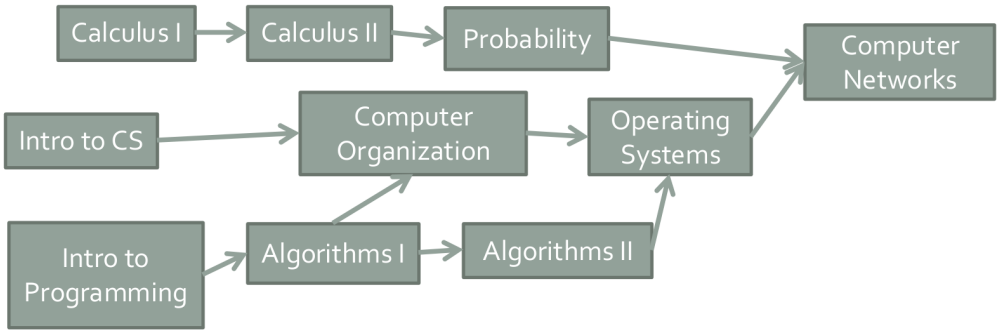
Applications of DFS

- What can it be used for?
- 1. Check if a undirected graph G is connected
- Review: what is a connected graph

- 2. List all connected component in an undirected graph G
- Review: what is a connected component?
- How to do it?



Topological Sort

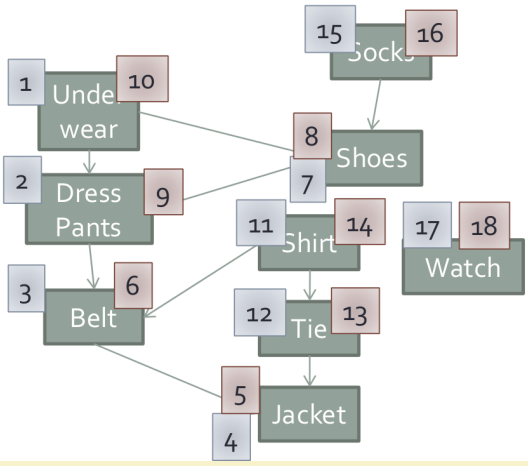


- When can you take course j? After completing all edge <i,j> in course i.
- All "activities" are represented by vertices
- Therefore, also called activity-on-vertex (AOV) network
- Q: How to find a valid course ordering?
- This ordering is called **topological order**
- To find a valid topological order, the graph must be a directed acyclic graph (DAG)

Topological Sort

$\Theta(V + E)$

- Call DFS(G) compute v.f for all vertices
- When each vertex is finished, insert it at the front of the linked list
- Return the linked list



Correctness Proof

- **Lemma 22.11:**
  G is acyclic if and only if the depth-first search of G has no back edges.
- Proof:
- (➔) Suppose there is a back edge <u,v>.
  Then v must be an ancestor of u in the depth-first forest.
- This means that there is another path from v to u along tree edges.
  Together with the edge <u,v>, this forms a cycle: v→u→v. This is a contradiction!
  Therefore, there should be no back edges.
- (➔) Suppose there is a cycle c in G.
- Let v be the first vertex in c to be discovered. Let <u,v> be an edge in c.
- When v is discovered, the path from v to u consists entirely of WHITE vertices.
  Therefore, by the White-Path Theorem, u will become a descendant of v.
- Thus, <u,v> is a back edge. This is a contradiction!
- Therefore, G has no cycle.

Correctness Proof

- Proof:
- Suppose we run DFS on the DAG G.
- Then, for any two vertices u and v, if G contains edge (u, v), then v.f < u.f. This can be explained as follows.
- When DFS explores edge (u, v), vertex v cannot be GRAY.
  If v were GRAY, then v would be an ancestor of u, and (u, v) would be a back edge.
- However, by the previous lemma, a DAG has no back edges. This is a contradiction!
- Therefore, v can only be BLACK or WHITE.
- If v is WHITE, then v becomes a descendant of u, so v.f < u.f.
- If v is BLACK, then v has already finished, so v.f has already been assigned. However, we are still exploring outward from u, so u.f has not been assigned yet.
  Therefore, v.f < u.f.
- Thus, for any edge (u, v), we have: u.f > v.f
- Vertices with smaller finishing times are placed later in the ordering.
  Therefore, whenever there is an edge (u, v), u always appears before v.

Correctness Proof

- Proof:
- Suppose we run DFS on a DAG G.
- For any two vertices u and v, if G contains edge (u, v), then v.f < u.f.
  This can be explained as follows.
- When DFS explores edge (u, v), v cannot be **GRAY**.
  If v were GRAY, then v would be an ancestor of u.
  In that case, (u, v) would be a back edge, which contradicts the previous theorem.
- Therefore, v can only be **BLACK** or **WHITE**.
- If v is WHITE, then v becomes a descendant of u, so v.f < u.f.
- If v is BLACK, then v has already finished, and v.f has already been assigned.
  However, DFS is still exploring outward from u, so u.f has not been assigned yet.
  Therefore, v.f < u.f.
- Thus, for every edge (u, v), we have **u.f > v.f**.
- Vertices with smaller f values are placed later in the ordering.
  Therefore, whenever there is an edge (u, v), u always comes before v.